FaceLit — propuesta técnica y arquitectura implementada

Corte de referencia: 27 de octubre de 2026.
Distinción: este documento registra la arquitectura descrita para el sistema actual; la propuesta de Render/Vercel de los documentos previos se conserva como antecedente histórico.

> **Revisión complementaria:** 4 de noviembre de 2026. Se inspeccionó el árbol de trabajo local de `FaceLit-Backend` y `FaceLit-DB`. Los cambios descritos en esta revisión aparecen sin seguimiento en Git en esos repositorios; por ello se documentan como trabajo presente en el árbol local, no como funcionalidad integrada en `main`, desplegada o aceptada.

Arquitectura lógica

FaceLit se organiza en cuatro componentes desplegables y una base compartida:

1. **Cliente FaceLit:** Expo/React Native. La versión web entrega pantallas y controla navegación, captura, estado y solicitudes HTTP.
2. **Backend FaceLit-Backend:** Java 17 y Spring Boot 3.4.1. Es un monolito modular organizado en autenticación, gestión académica, ambientes/sesiones, facial/asistencia, notificaciones y módulos compartidos.
3. **Servicio micro-facial:** Python 3.11 y FastAPI. Ejecuta detección, generación de embeddings, evaluación de retos y búsqueda de coincidencia para el contexto de sesión.
4. **FaceLit-DB:** PostgreSQL 16 con pgvector y Liquibase. El repositorio administra esquemas, datos iniciales y cambios

versionados; no es un servicio HTTP separado.

Conexiones

- Navegador → Caddy por HTTPS para servir la web y para solicitudes HTTP/WebSocket.
- Caddy → backend Spring Boot para API y canal WebSocket.
- Backend → servicio facial por HTTP en la red privada de Compose, con una clave de servicio mantenida en el servidor.
- Backend → PostgreSQL mediante JPA/JDBC.
- Servicio facial → PostgreSQL mediante SQLAlchemy asíncrono y asyncpg.
- Liquibase → PostgreSQL antes de que el backend valide el esquema.

Autenticación, registro y consentimiento (revisión local del 4 de noviembre)

El backend conserva la autenticación como módulo interno del monolito. En el árbol local inspeccionado se encontró un controlador de registro bajo `/api/auth` con tres operaciones públicas: `POST /register`, `POST /verify-email` y `POST /resend-code`. El registro crea la cuenta en estado pendiente, asigna el rol inicial `APPRENTICE`, guarda una credencial con contraseña codificada y envía un código de verificación de seis dígitos con expiración de cinco minutos. La verificación activa la cuenta de una persona adulta; para una persona menor, marca el correo como verificado y mantiene la cuenta pendiente de consentimiento. El reenvío invalida el código previo.

También hay componentes del dominio legal para aceptación de términos y aviso de privacidad, consentimiento, verificación del consentimiento y datos del acudiente. En la base de datos local aparece la migración `025_terms_acceptance_notice_version`, que agrega una versión del aviso a cada aceptación y una unicidad por usuario y versión. La presencia de estos modelos y migraciones en el

árbol no demuestra por sí sola que el flujo legal completo esté conectado, probado de extremo a extremo o desplegado.

El envío de códigos depende del servicio de correo del backend. La configuración y la entrega efectiva de correo deben documentarse y verificarse en el entorno correspondiente antes de considerar operativo el registro. La aceptación de términos y el consentimiento del acudiente son flujos distintos: no se debe describir la verificación del correo como autorización legal.

La guía describe el backend como punto de coordinación del flujo de asistencia. El frontend actual envía registro facial al endpoint del backend bajo /api/facial/embeddings; el servicio Java puede invocar el servicio FastAPI. En el código del frontend revisado, la clave facial no se usa en la llamada del flujo, pero el README de ese repositorio aún menciona variables EXPO_PUBLIC de integración facial heredadas. Eliminar esas referencias de configuración/documentación y revisar el bundle es una tarea de seguridad pendiente; no afirmar que toda credencial histórica fue retirada hasta comprobarlo.

Flujo principal de asistencia

1. El cliente envía por HTTPS la captura y el identificador/contexto de sesión al backend.
2. El backend valida identidad, rol, alcance y contexto de sesión.
3. Para la evaluación facial, el backend llama al servicio FastAPI y espera su respuesta.
4. El servicio decodifica las imágenes, detecta el rostro, genera características y valida la secuencia de retos; consulta candidatos autorizados en PostgreSQL.
5. El backend aplica reglas de horario, entrada/salida y duplicados.
6. Si la operación cumple las reglas, se persiste el evento y se responde al cliente.

La web no debe conectarse directamente a PostgreSQL ni exponerse a la clave interna del servicio facial. El servicio facial no necesita un puerto público para el flujo documentado.

Capas y responsabilidades

Componente	Entrada/ presentación	Aplicación/ negocio	Datos e integraciones
FaceLit	Rutas, pantallas y componentes	Hooks coordinan acciones, formularios y estado	Servicios HTTP y stores
Backend	Controladores y DTO	Servicios y políticas de autorización	Repositorios JPA/JDBC, correo, WebSocket y cliente facial
Micro-facial	Rutas FastAPI y esquemas Pydantic	Servicio de embedding y política de liveness	SQLAlchemy, PostgreSQL, trabajador de inferencia
FaceLit-DB	No aplica como capa web	Reglas de esquema, relaciones y restricciones	SQL, changelogs, índices y Liquibase

Las tres capas son una separación lógica dentro de las aplicaciones, no tres servidores por aplicación. El backend sigue siendo un monolito modular.

Tecnologías y motivo de uso

Tecnología	Responsabilidad documentada
Expo, React Native y TypeScript/JavaScript	Interfaz, navegación y acceso a cámara

Java/Spring Boot	API, reglas de negocio y coordinación transaccional
Spring Security/JWT	Identidad y autorización de solicitudes
Python/FastAPI/Pydantic	API facial y contratos de solicitudes/respuestas
InsightFace/ONNX Runtime	Detección y representación facial
MediaPipe	Puntos/métricas faciales para evaluar movimientos
PostgreSQL/pgvector	Datos relacionales y vectores faciales
SQLAlchemy/asyncpg	Acceso asíncrono a PostgreSQL desde Python
Liquibase	Evolución reproducible del esquema
Docker Compose/Caddy	Servicios privados en la red del VPS y terminación/proxy HTTPS
WebSocket	Actualizaciones seleccionadas sin depender solo de refrescar la página

Decisiones y límites

- Los módulos del backend no son microservicios independientes.
- Backend y facial comparten la base; la independencia de datos no está implementada.
- No se ha medido rendimiento bajo carga ni tiempo máximo garantizado.
- La existencia de interfaces, capas o tests no demuestra que cada requisito esté aceptado en producción.
- El flujo facial y de asistencia físico debe validarse con la versión desplegada y cámara real.
- Los cambios de registro, consentimiento y versión del aviso vistos el 4 de noviembre aún no están confirmados en Git en los repositorios de aplicación y base de datos; no atribuirlos al

despliegue sin una revisión posterior.

Referencias de código

SHA de referencia de la arquitectura base: FaceLit 5f7d6fe, FaceLit-Backend 2ab0bcd, micro-facial 1e12c6c y FaceLit-DB 41a063f. Ejemplos para la revisión local del 4 de noviembre: `FaceLit-Backend/src/main/java/com/FaceLit/backend/auth/controller/security/RegisterController.java`, `auth/service/serviceImpl/security/RegisterServiceImpl.java`, `auth/model/legal/` y `FaceLit-DB/01_ddl/04_alter/025_terms_acceptance_notice_version.sql`. La revisión local no tiene SHA de commit asociado.